

2024 Fall OOP Assignment Report

과제 번호 : 5

학번 : 20230673

이름 : 전재영

Povis ID : jason9751

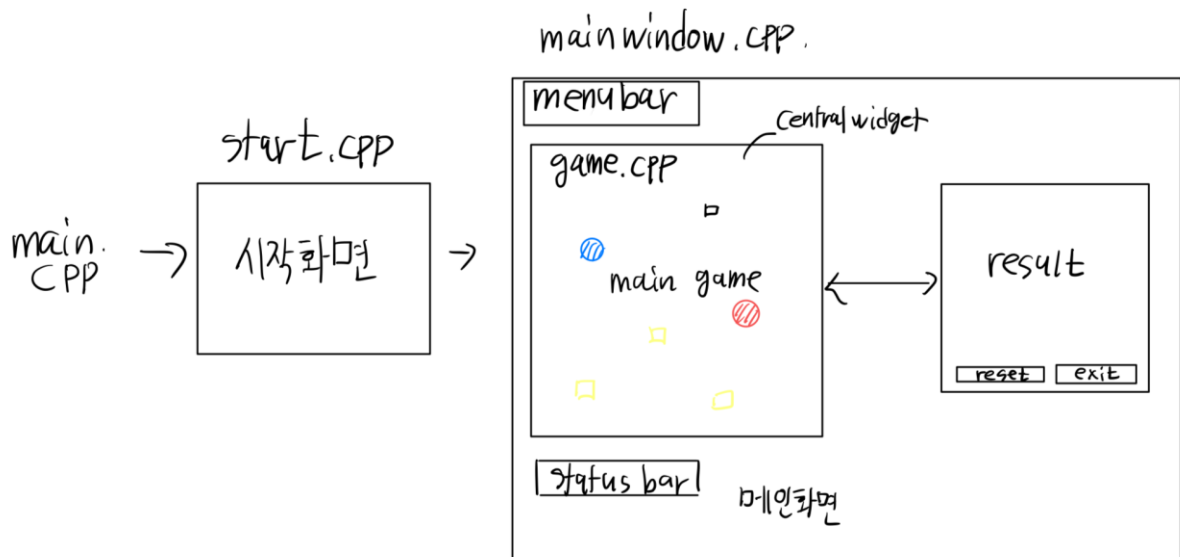
명예서약 (Honor Code)

나는 이 프로그래밍 과제를 다른 소스를 인용없이 참고하거나,
다른 사람의 부적절한 도움을 받지 않고 완수하였습니다.
I completed this programming task without using any uncited sources
or receiving improper help from others.

1. 프로그램 구조 및 알고리즘

- 프로그램 구조

- Qt를 이용해 snake game을 만드는 프로그램이다.
- 대략적인 class의 구조도를 그리면 다음과 같다.



- `main.cpp`, `start.cpp(h)`, `mainwindow.cpp(h)`, `game.cpp(h)` 파일이 있다.
 - `Main.cpp`에서는 `StartWindow` 만들어 시작화면을 보여준다.
 - `StartWindow`에서 시작을 누르면, `MainWindow`를 만들어 메인 화면으로 이동한다.
 - 메인 화면에서 화면을 구성하는 `menubar`, `status` 위젯을 넣고, `Game`을 만들어서, 넣

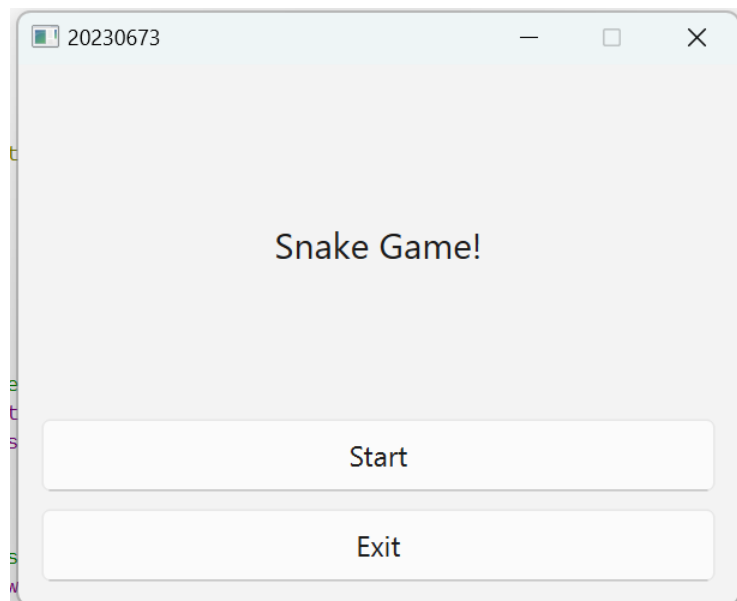
는다.

- Timer를 이용해 Game에서의 결과를 mainwindow에 업데이트한다.
- 만약 게임이 종료되면 result 화면을 표시한다.

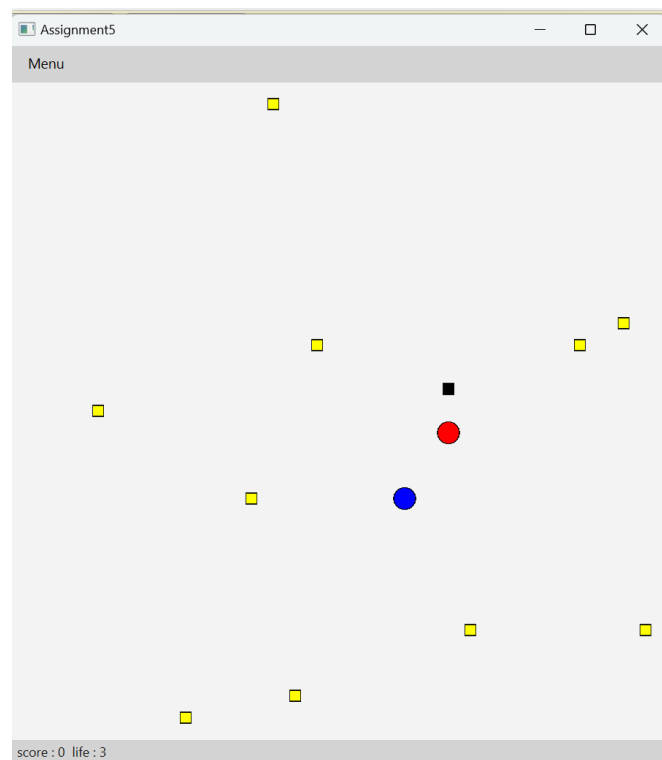
- 세부 알고리즘

- StartWindow는 QVBoxLayout, QLabel, QPushButton으로 이루어져 있다. 제목을 QLabel에 적고, QVBoxLayout에 넣는다. QPushButton을 이용해 startbutton과 exitbutton을 만들어, connect 함수를 이용해, 버튼을 클릭했을 때의 행동을 지정한다. Startbutton을 누르면, openGameWindow() 함수를 실행해, 메인 화면을 만들어 이동하고, exitbutton을 누르면 종료한다.
- MainWindow는 대부분의 프로그램을 구성하는 메인 UI이다. Game을 만들어서 게임을 시작하고, set_up()를 통해 QMenuBar, QMenu, statusBar를 만든다. Game은 setCentralWidget을 이용해 MainWindow의 중간에 배치한다.
- Game widget은 600 * 600의 사이즈를 가진다. Game class가 만들어졌을 때, Snake, Enemy snake, coin, food의 위치를 결정한다. 20 ~ 580를 20 간격으로 x와 y의 값을 결정한다. 만약 다른 아이템과 위치가 겹칠 수도 있으므로, 다른 QList를 순회해서 위치가 겹치는지를 확인한 후, 위치를 정한다.
- Timer를 이용해 200ms마다 Enemy snake의 위치를 업데이트 한다. isNotMovable(), isThisDirectionNotMovable() 을 이용해 이동할 수 없는 방향을 제외하고, 랜덤한 위치로 한칸 이동한다.
- Player Snake는 키 입력을 이벤트로 받아서 이동한다. keyPressEvent을 이용하며, Qt::Key_W를 입력받으면, dx = 0; dy = -20; move = 2 (위쪽 방향) 등 으로 설정하는 방식이다.
 - 만약 food와 위치가 같게 된다면, List에서 push와 pop을 하는 과정에서 pop을 하는 과정을 생략해 암시적으로 Snake의 길이를 길게 한다. 그 후, food를 랜덤한 위치에 다시 배치한다.
 - 만약 coin과 위치가 같게 된다면, Coin을 List에서 삭제하고, Score를 1 증가시킨다.
 - 만약 죽었을 경우, 즉 isThisDirectionNotMovable()이나, isThisDirectionSnake-NotMovable이 참일 경우는 Snake의 List를 초기화하고, 다시 랜덤한 위치에 Snake를 배치한다. 그리고 player_life를 1 감소시킨다.
- paintEvent를 이용해 맵을 그린다.
- 위의 Game에서의 값을 이용해 MainWindow에서도 Timer를 이용해 표시하는 값을 업데이트 한다. Statusbar를 업데이트 해서, 지금 내가 몇점이고, 몇 목숨이 남았는지를 표시한다.

- 만약 Life가 0이 되거나, 메뉴창에서 Stop을 누른다면, end_game()을 실행해, QMessageBox을 띄운다. Score를 띄우고, StandardButtons으로 있는 Reset과 Close의 기능을 넣은 버튼을 넣었다. 그래서, 다시 프로그램을 시작하거나, 종료할 수 있다.
- 실행 방법 및 예제
 - Qt Creator를 이용해 실행을 했다.

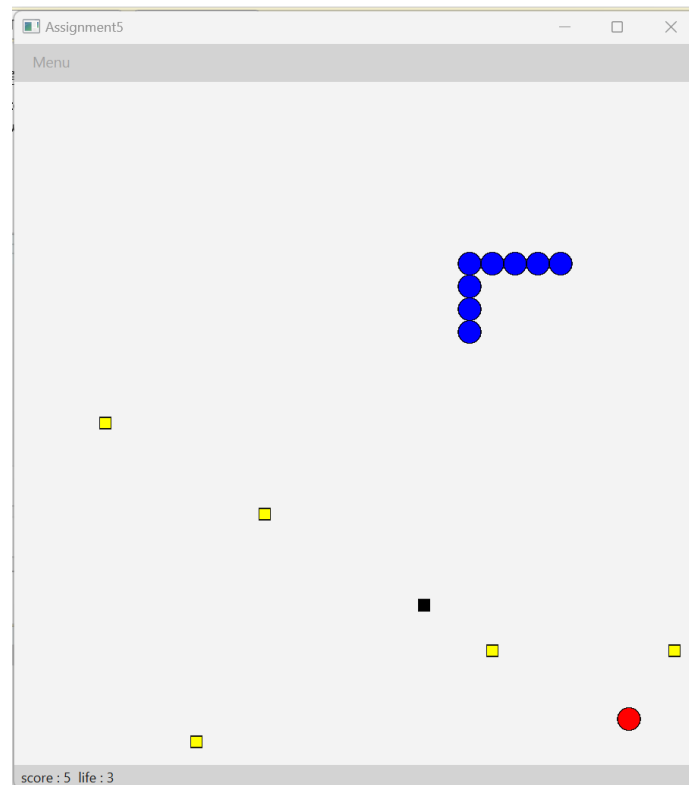


- 게임의 시작화면이다.

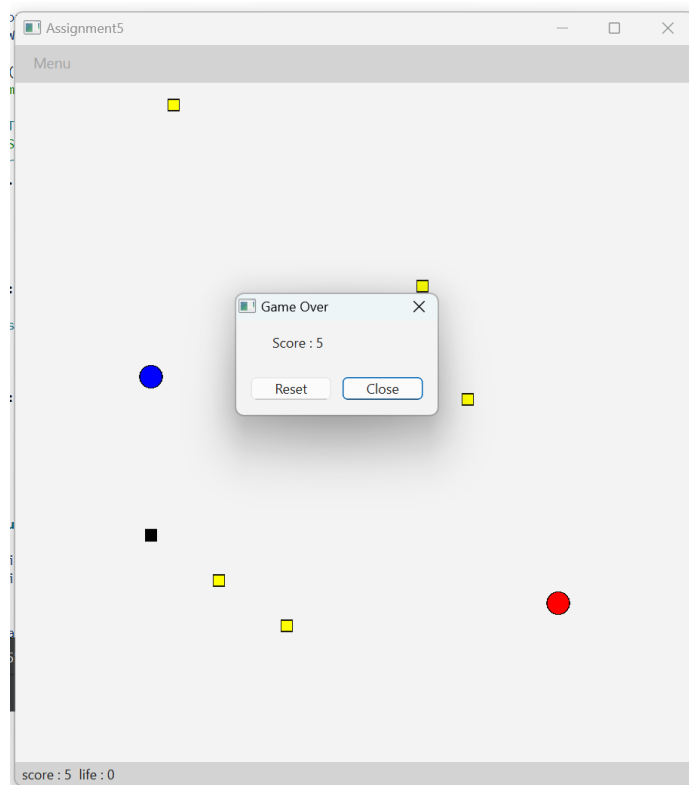


- 게임 화면은 다음과 같다. 위의 메뉴바와 상태바를 회색으로 색칠해 게임 화면과 구분하

였다.



- 코인을 먹으면 score가 증가하고, 음식을 먹으면 길이가 늘어나는 것을 확인 할 수 있다.

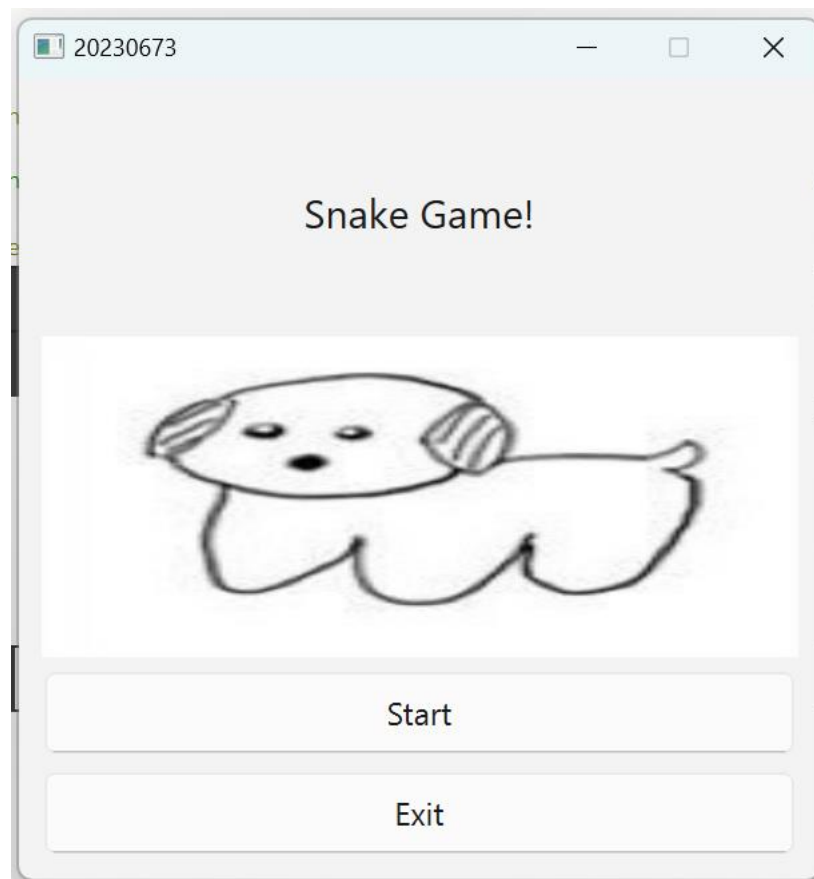


- Life가 0이 되거나, 메뉴에서 Stop을 선택하면, Game Over Message Box를 보인다.

2. 토론 및 개선 (20%)

- 토론

- 이번 과제를 통해 Qt를 이용한 GUI 구현과, 객체 지향 프로그래밍에서의 상속 관계에 대해 공부할 수 있었다.
- 이미지를 넣어서 시작 화면을 구성하려고 했는데, 잘 수행되지 않았다. 프로그램이 컴파일되어서 실행되는 위치는 `"/build/Desktop_Qt_6_8_1_MinGW_64_bit-Debug/"` 인데, 사진 파일의 위치는 build 파일 위의 있는 프로젝트 파일에 위치해서 찾지 못한 것 같다. 만약 `"C:\Users\WWjjong\WW...WWAssignment5\WWjjong22.jpg"`와 같이 절대 경로를 이용하면 정상적으로 이미지 파일을 불러 올 수 있었다. 어떻게 해야 상대 경로의 이미지 파일을 불러올 수 있는지 조사할 필요가 있어 보인다.



- 개선 방향

- 가끔 Player와 Enemy가 겹치는 경우가 있는데, Enemy는 200ms로 Timer를 이용해 업데이트를 하고, Player는 keyEvent를 이용해 업데이트를 하기 때문에, 이 시간의 차이 때문에 문제가 발생하는 것 같다. 이를 개선하기 위해서는 Update Manager를 만들어 모든 업데이트 사항을 관리하게 만들면 좋을 것 같다. Update Manager에서 일정 시간마다 빠르게 업데이트를 수행해 만약 Player와 Enemy가 변한다면, 각각 처리하는 것이 아니라, Manager에서 같이 처리하고 동시에 업데이트 하는 방식으로 변환

하면, 이러한 오류를 줄일 수 있을 것 같다.

- Menubar와 StatusBar를 각각의 Class로 만든 것이 아니라, MainWindow에서 직접 들어서 사용했는데, 이를 좀 더 세분화하여서 각각의 클래스를 만들어 관리하는 것이 좋아보인다. 만약 클래스를 나눠서 SIGNAL을 이용해 관리한다면, MainWindow에서 Timer를 통해 업데이트 하는 과정이 없이, Game에서 업데이트 한 내용을 MainWindow에서 SIGNAL로 받아 StatusBar로 전달하는 식으로 구현해, 프로그램의 실행 속도를 더 높일 수 있을 것 같다.

3. 참고 문헌 (10%)

- 참고 문헌

Qt StatusBar : <https://blog.naver.com/hextrial/221109244283>

Qt Image : <https://blog.naver.com/hextrial/221109232458>

절대/상대 경로 :

<https://velog.io/@bami/%ED%8C%8C%EC%9D%BC-%EA%B2%BD%EB%A1%9C-%EC%A0%88%EB%8C%80-%EA%B2%BD%EB%A1%9C%EC%99%80-%EC%83%81%EB%8C%80-%EA%B2%BD%EB%A1%9C>